1. Verificación del Desacoplamiento de Capas

Ejecuta la inspección de historial de la imagen:

Bash
docker history --human --format "table {{.Size}}\t{{.CreatedBy}}" eco-air-backend:1.0.0

2. Inspección del Proceso del Host (Aislamiento de Privilegios)

Inicia el contenedor en modo detached:

Bash
docker run -d --name eco_air_active -p 8080:8080 eco-air-backend:1.0.0
Consulta la tabla de procesos internos:

Bash
docker top eco_air_active -o pid,user,comm

- **Evidencia:** Demostrar que el comando java no se ejecuta como root, sino bajo el identificador de sensor_user.

```
PS C:\Users\WILLLIAM\Desktop\taller-docker-semana13> docker run -d --name eco_air_active -p 8080:8080 eco-air-backend:1.0.0
ed14950511dd48c03307a3403eb01fe147dc8fd0eede4dfb209cab270370946a
PS C:\Users\WILLLIAM\Desktop\taller-docker-semana13> docker top eco_air_active -o pid,user,comm
PID                 USER                COMMAND
941                 10001               java
```

3. Test de Penetración: Comprobación de Inmutabilidad de Runtime

Intenta escribir un script malicioso en un directorio del sistema:

Bash
docker exec eco_air_active touch /etc/crontab_fake

- **Evidencia:** Colocar evidencia

```
PS C:\Users\WILLLIAM\Desktop\taller-docker-semana13> docker exec eco_air_active touch /etc/crontab_fake
touch: cannot touch '/etc/crontab_fake': Permission denied
```

4. Auditoría de Artefactos de Desarrollo

Bash
docker exec eco_air_active which mvn
docker exec eco_air_active find / -name "*.java" 2>/dev/null

- **Evidencia:** Ambas salidas y explicar el resultado

```
PS C:\Users\WILLLIAM\Desktop\taller-docker-semana13> docker exec eco_air_active which mvn
PS C:\Users\WILLLIAM\Desktop\taller-docker-semana13> docker exec eco_air_active sh -c 'find / -name "*.java" 2>/dev/null'
```

El resultado vacío en los dos comandos confirma que aislamos por completo el entorno de desarrollo del de producción. Al verificar que no existe ni el binario de Maven ni ningún rastro del código fuente original dentro del contenedor activo, asi que un atacante no tendría herramientas locales para modificar la aplicación o inspeccionar nuestra lógica. La imagen cumple de forma limpia con el principio de mínimo privilegio.

Reto 3: Calibración de cgroups y Captura del Asesino OOM

Los sensores ambientales envían ráfagas continuas de métricas. En este ejercicio se someterá el contenedor a asfixia de memoria para validar el comportamiento del kernel.

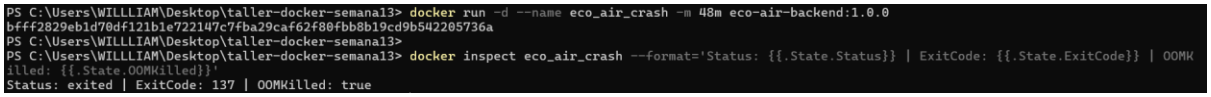
1. Inducción del Colapso por Memoria (*OOMKilled*)

Ejecuta una instancia asignándole un límite de memoria de tan solo **48 MB** (insuficiente para levantar el framework Spring Boot con Hibernate):

Bash
docker run -d --name eco_air_crash -m 48m eco-air-backend:1.0.0
Espera 10 segundos y extrae el estado de terminación del contenedor:

Bash
docker inspect eco_air_crash --format='Status: {{.State.Status}} | ExitCode: {{.State.ExitCode}} | OOMKilled: {{.State.OOMKilled}}'

- **Evidencia obligatoria:** Adjuntar la salida de la consola explicar resultado.



Al limitar drásticamente la memoria RAM del contenedor a solo 48 MB usando el parámetro -m 48m, provocamos deliberadamente un colapso en el arranque de la aplicación. Como el framework (Spring Boot con Hibernate) y la máquina virtual de Java (JVM) requieren un consumo mínimo muy superior para inicializar sus componentes y levantar el backend, el contenedor excedió inmediatamente la cuota asignada de hardware.

2. Calibración del Límite Operativo Mínimo

Modifica el parámetro -m de forma iterativa (ej. 256m, 384m, 512m) hasta hallar el umbral mínimo exacto en el que:

1. El contenedor inicie correctamente.
2. Los logs confirmen que la aplicación arrancó en menos de 15 segundos:

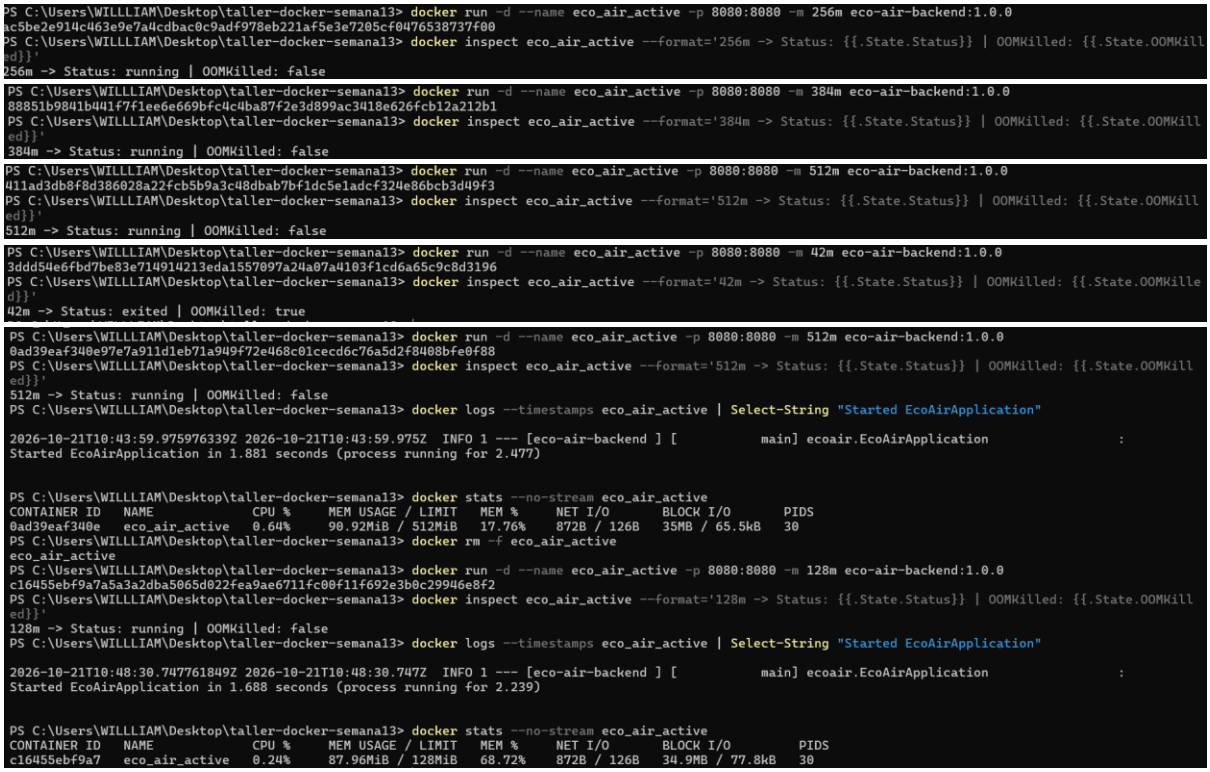
   Bash

   docker logs --timestamps eco_air_active | grep "Started EcoAirApplication"
3. El profiling de memoria en caliente se mantenga estable:

   Bash

   docker stats --no-stream eco_air_active

- **Consigna:** Adjunta la tabla resultante de docker stats indicando: % de CPU, MEM USAGE / LIMIT y MEM %.



Finalmente, se realizó la calibración de cgroups sometiendo el contenedor a pruebas de estrés de memoria. Se demostró que un límite de 48 MB induce un colapso inmediato por parte del kernel (OOMKilled: true). Mediante pruebas iterativas, se determinó que 128 MB es el umbral operativo mínimo estable para el arranque de Spring Boot, garantizando un funcionamiento óptimo con un consumo en reposo de ~88 MB (68.72% del límite asignado).